SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: MLA03 COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

(10 Marks)

2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

(10 Marks)

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

(10 Marks)

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

(10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of

deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and εgreedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

EXPERIMENT 1:

GRID WORLD NAVIGATION USING REINFORCEMENT LEARNING

AIM:

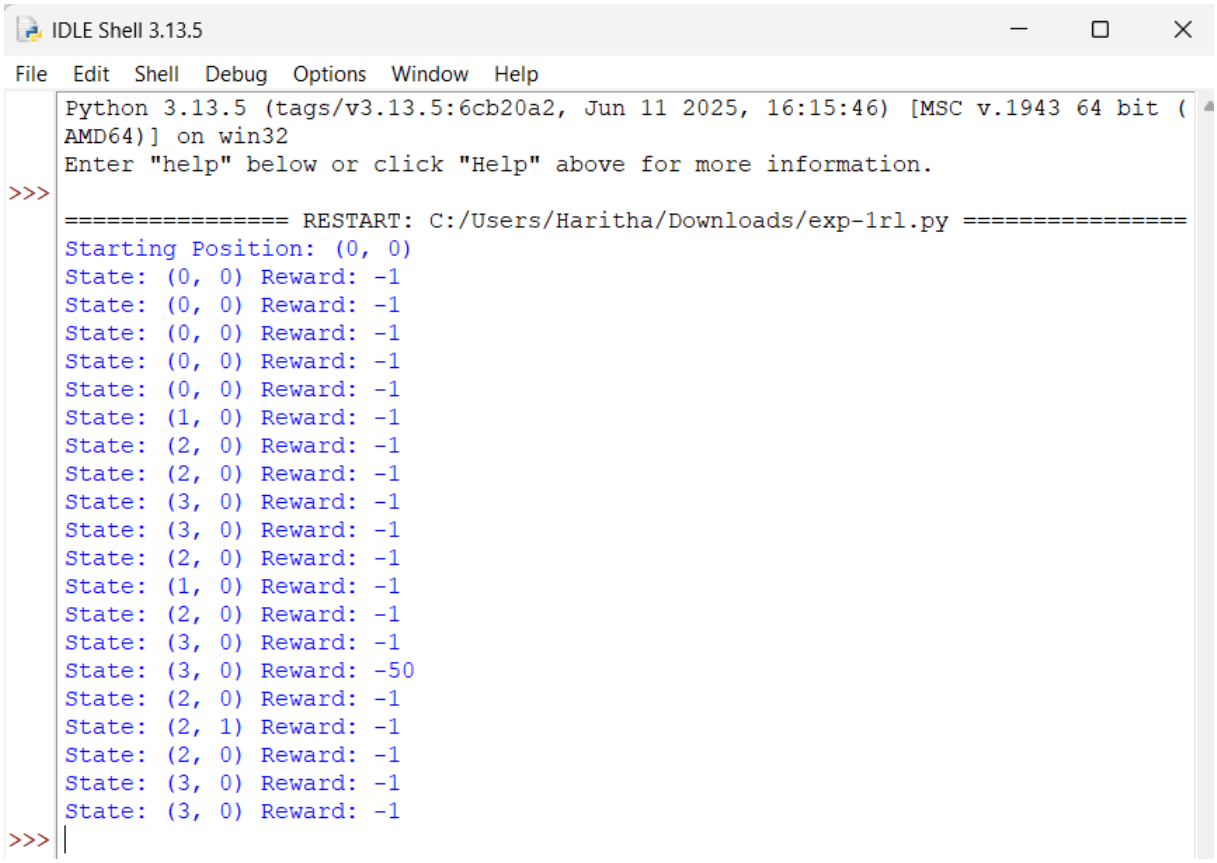
To develop a Python-based Reinforcement Learning model using a Grid World environment where an agent learns to reach the goal while avoiding obstacles using a reward-based navigation strategy.

PROGRAM

```
import random
grid_size = 5
goal = (4, 4)
obstacles = [(1, 2), (2, 2), (3, 1)]
state = (0, 0)
actions = {
    0: (-1, 0),   # Up
    1: (1, 0),    # Down
    2: (0, -1),   # Left
    3: (0, 1)     # Right
}
print("Starting Position:", state)
for step in range(20):
    action = random.randint(0, 3)
    move = actions[action]
    new_state = (
        max(0, min(grid_size - 1, state[0] + move[0])),
        max(0, min(grid_size - 1, state[1] + move[1]))
    )
    if new_state in obstacles:
        reward = -50
    elif new_state == goal:
        reward = 100
        state = new_state
        print("Goal Reached!")
```

```
        break
    else:
        reward = -1
        state = new_state
    print("State:", state, "Reward:", reward)
```

OUTPUT:



EXPERIMENT 2: E-GREEDY MULTI-ARMED BANDIT

AIM

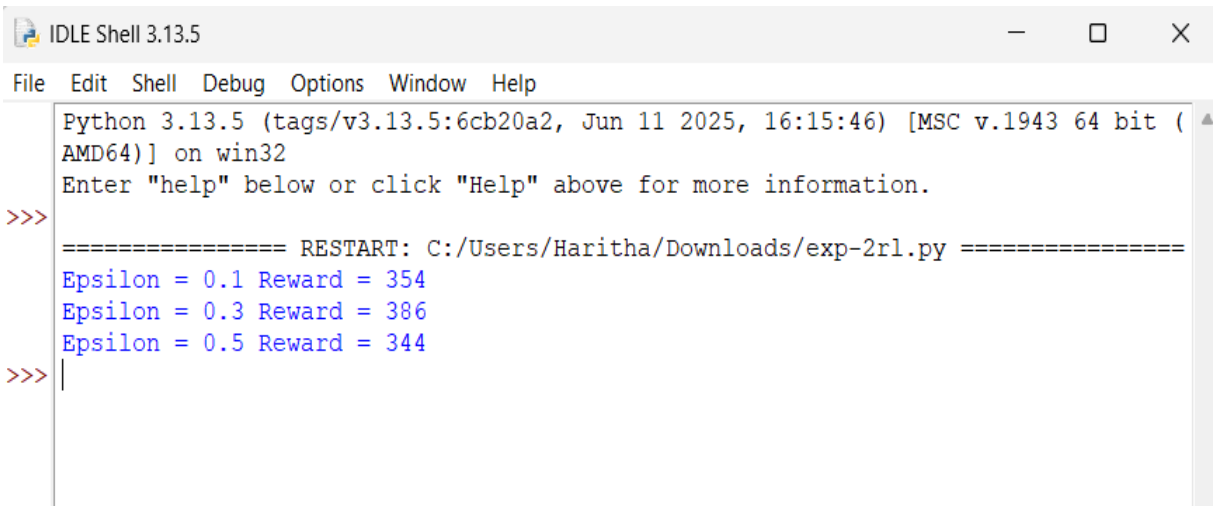
To implement the ε-Greedy Multi-Armed Bandit algorithm in Python to maximize rewards within a budget of 500 trials and compare different ε values.

PROGRAM:

```
import random
import numpy as np


arms = [0.2, 0.5, 0.7, 0.4, 0.9]
trials = 500
```

```
def epsilon_greedy(epsilon):
    counts = np.zeros(len(arms))
    values = np.zeros(len(arms))
    total_reward = 0

    for i in range(trials):

        if random.random() < epsilon:
            arm = random.randint(0, 4)
        else:
            arm = np.argmax(values)

        reward = 1 if random.random() < arms[arm] else 0

        counts[arm] += 1
        values[arm] += (reward - values[arm]) / counts[arm]
        total_reward += reward

    return total_reward

for e in [0.1, 0.3, 0.5]:
    print("Epsilon =", e, "Reward =", epsilon_greedy(e))
```

OUTPUT:



EXPERIMENT 3: Markov Decision Process (MDP)

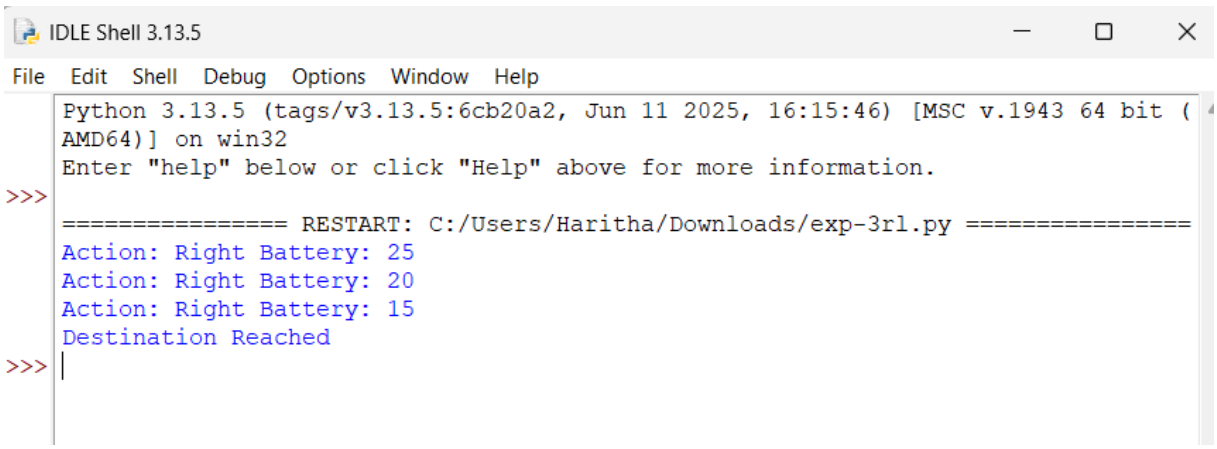
AIM

To design a Markov Decision Process for an autonomous robot that reaches a destination while minimizing energy consumption.

PROGRAM:

```
states = ["Start", "A", "B", "Goal"]

actions = ["Up", "Down", "Left", "Right"]

transition = {
    ("Start", "Right"): "A",
    ("A", "Right"): "B",
    ("B", "Right"): "Goal"
}

battery = 30

for action in ["Right", "Right", "Right"]:

    battery -= 5
    print("Action:", action, "Battery:", battery)

print("Destination Reached")
```

OUTPUT:



EXPERIMENT 4: Value Iteration using TensorFlow/Keras

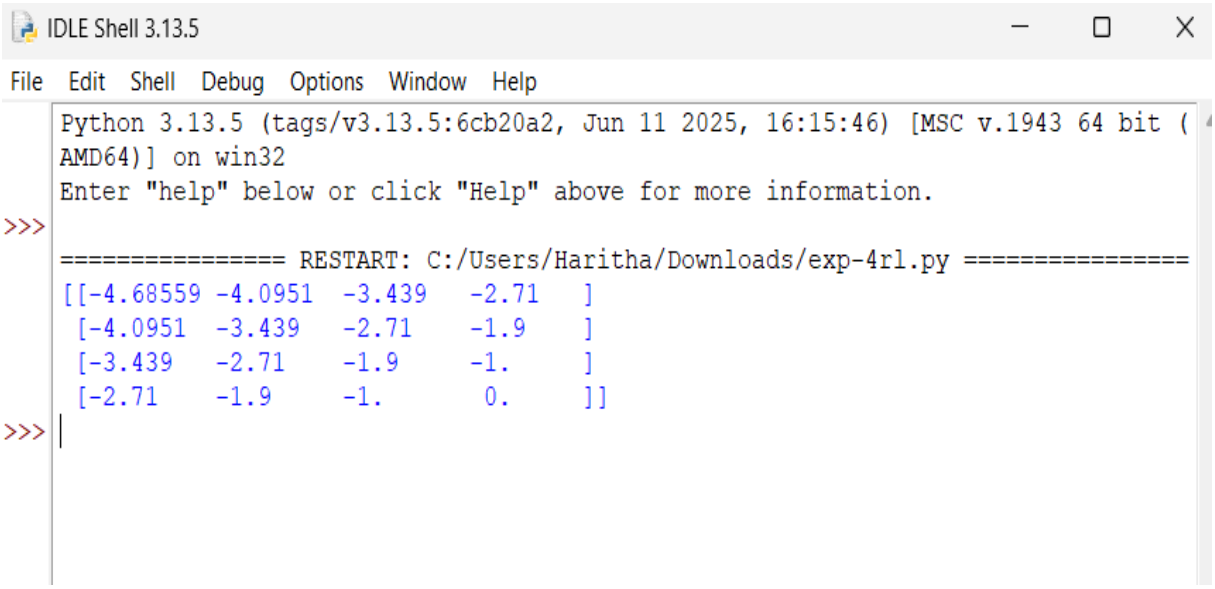
AIM

To implement the Value Iteration algorithm using Python and TensorFlow/Keras to determine the optimal policy for a constrained Grid World using the Bellman equation.

PROGRAM:

```
import numpy as np
gamma = 0.9
V = np.zeros((4, 4))
goal = (3, 3)
for iteration in range(20):

    newV = V.copy()

    for i in range(4):
        for j in range(4):

            if (i, j) == goal:
                continue

            values = []

            for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:

                ni = max(0, min(3, i + dx))
                nj = max(0, min(3, j + dy))

                values.append(-1 + gamma * V[ni][nj])

            newV[i][j] = max(values)

    V = newV

print(V)
```

OUTPUT:



EXPERIMENT 5: WAREHOUSE ROBOT USING REINFORCEMENT LEARNING

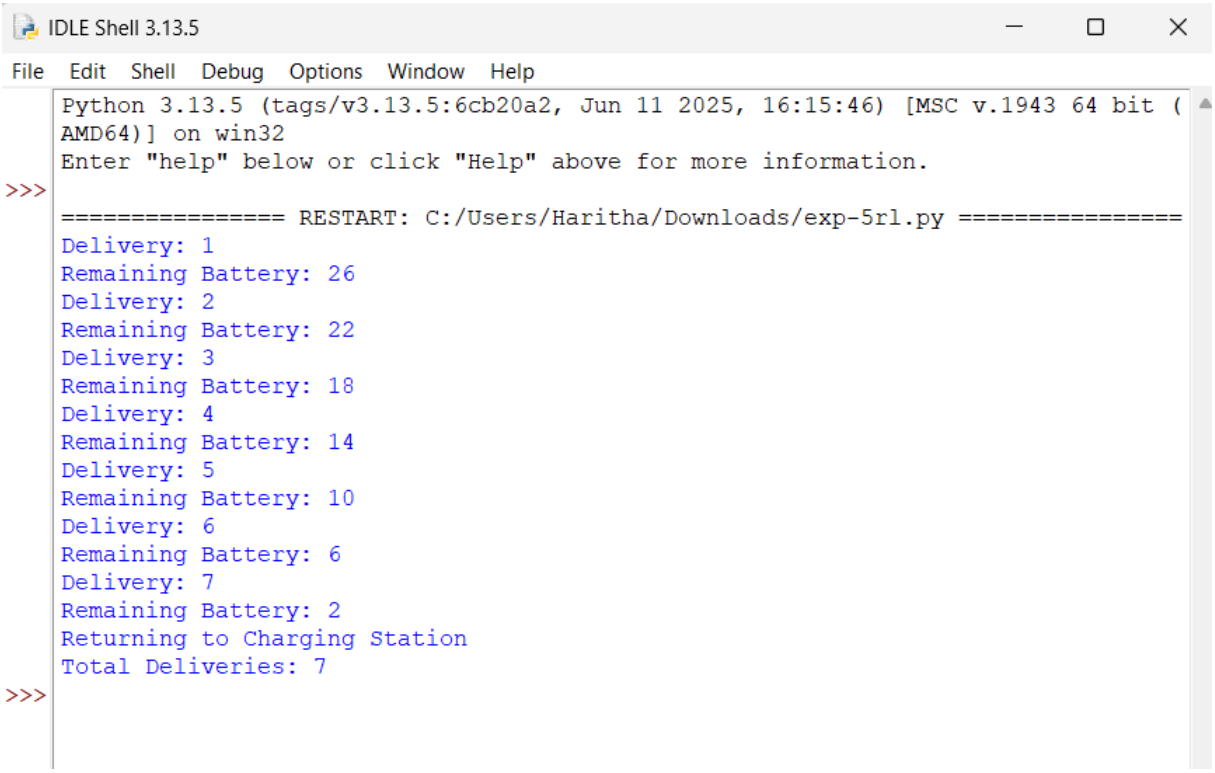
AIM:

To design a Reinforcement Learning framework that enables a warehouse robot to maximize the number of deliveries without exceeding a battery capacity of 30 minutes.

PROGRAM:

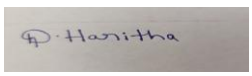
```
battery = 30
deliveries = 0

while battery >= 5:

    deliveries += 1
    battery -= 4

    print("Delivery:", deliveries)
    print("Remaining Battery:", battery)

print("Returning to Charging Station")
print("Total Deliveries:", deliveries)
```

OUTPUT:



Code of Conduct:

*I **Dasari Haritha(192425254)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION

Criteria	Weightag	Level 4	Level 3	Level 2	Level 1

	e	(Exemplary)	(Proficient)	(Developing)	(Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided